

Final_Project_IL

PP

2026-07-09

Contents

1	Introduction	1
1.1	Overview	1
2	Method	2
2.1	Data cleaning	2
2.2	Data analysis preparation	2
2.2.1	library loading	2
2.2.2	Data download from Git/Kaggle	2
2.2.3	Basic data exploration	2
2.2.4	Exploration on NA values	3
2.2.5	Creation of final holdout set and separate train/test sets afterwards	3
2.2.6	Further data exploration using visualisation	4
3	Results	11
3.1	Handling of NA	11
3.2	Model 1: logistic regression (glm)	11
3.3	Model 2: KNN	12
3.4	Model 3: gamLoess	14
3.5	Model 4: Random Forest	16
3.6	Summary table of individual models	16
3.7	Ensemble with stacked models	17
3.8	Test on final hold out set	20
4	Conclusion	23
5	References	23

1 Introduction

1.1 Overview

Liver disease is 11th leading cause of death in Australia, and account for 2 million deaths per annum world-wide, which is 4% of all deaths.(1,2) This highlights the importance of easy, accessible and scalable detection for those at risk. Liver disease can be divided into cholestatic and hepatocellular disorders - one that is caused by obstruction and the other by direct damage to the liver.(1) Currently liver function test (LFT) based on blood test is used in routine clinical practice to detect liver disease.

The purpose of this project is to develop a liver disease prediction models based on a dataset which contains 416 liver patient records and 167 non liver patient records collected from North East of Andhra Pradesh, India. This data has been made publicly available via Kaggle (more information is available below in Methods).

Main method is based on machine learning techniques such as linear regression, gamLoess, KNN and random forest. Ensemble method has also been trialed.

The English (Australian) style was used throughout this documentation given author's geographic background.

2 Method

2.1 Data cleaning

Tidy data has been already provided and this step was not required.

2.2 Data analysis preparation

2.2.1 library loading

```
## Loading required package: tidyverse

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.3      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: caret

## Warning: package 'caret' was built under R version 4.4.1

## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##      lift
```

2.2.2 Data download from Git/Kaggle

Original data can be accessed via <https://www.kaggle.com/uciml/indian-liver-patient-records>. To assist with code access without Kaggle account, I have uploaded the dataset onto my Github account for easier access.

```
url <- "https://raw.githubusercontent.com/pswpark/Final\_Project\_IL/main/indian\_liver\_patient.csv"

liver_data <- read.csv(url)
```

2.2.3 Basic data exploration

This identifies that Dataset column contains integers, instead of factors, which would be more useful given categorisation task.

```
names(liver_data) #note "Dataset": field used to split the data into two sets (patient with liver disease, patient without liver disease)
str(liver_data)

head(liver_data)
```

```

# Factorisation and creation of disease column instead of Dataset
liver_data <- liver_data %>%
  mutate(Disease = as.factor(Dataset),
         Gender = as.factor(Gender)) %>%
  select(-Dataset)

# R variable name error will occur if we use 1 and 2 as factor names when we create ensemble.
liver_data$Disease <- factor(
  liver_data$Disease,
  levels = c(1, 2),
  labels = c("Yes", "No")
)

levels(liver_data$Disease) #correct assignment of 1 and 2.

```

2.2.4 Exploration on NA values

There are 4 NA values in the albumin-globulin ratio column. The impact of NA are likely to be minimal.

```

summary(liver_data)
sum(is.na(liver_data))
colSums(is.na(liver_data)) # 4 NA values in AGRatio column

which(is.na(liver_data), arr.ind = TRUE) #row numbers

liver_data[c(210,242,254,313),] #the NA rows visualised with other variables

```

##	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase
## 210	45	Female	0.9	0.3	189
## 242	51	Male	0.8	0.2	230
## 254	35	Female	0.6	0.2	180
## 313	27	Male	1.3	0.6	106

##	Alamine_Aminotransferase	Aspartate_Aminotransferase	Total_Protiens	Albumin
## 210	23		33	6.6 3.9
## 242	24		46	6.5 3.1
## 254	12		15	5.2 2.7
## 313	25		54	8.5 4.8

##	Albumin_and_Globulin_Ratio	Disease
## 210	NA	Yes
## 242	NA	Yes
## 254	NA	No
## 313	NA	No

2.2.5 Creation of final holdout set and separate train/test sets afterwards

```

# Final hold-out test set will be 15% of the total data based on Disease column
set.seed(1, sample.kind="Rounding")

## Warning in set.seed(1, sample.kind = "Rounding"): non-uniform 'Rounding'
## sampler used

test_index <- createDataPartition(y = liver_data$Disease, times = 1, p = 0.15, list = FALSE)
liv <- liver_data[-test_index,]
final_holdout_test <- liver_data[test_index,]

```

```

# Splitting of train/test sets with 15% in the test set

set.seed(1, sample.kind="Rounding")

## Warning in set.seed(1, sample.kind = "Rounding"): non-uniform 'Rounding'
## sampler used

test_index <- createDataPartition(y = liv$Disease, times = 1, p = 0.15, list = FALSE) #note the data is
train_set <- liv[-test_index,]
test_set <- liv[test_index,]

# table summary of datasets we have created:
datasets <- list(liver_data, liv, train_set, test_set, final_holdout_test)

dim_table <- data.frame(
  Dataset = c("liver_data", "liv", "train_set", "test_set", "final_holdout_test"),
  do.call(rbind, lapply(datasets, dim))
)

names(dim_table)[2:3] <- c("Rows", "Columns")

```

```
dim_table
```

```
##           Dataset Rows Columns
## 1      liver_data  583      11
## 2           liv   494      11
## 3      train_set  419      11
## 4      test_set   75      11
## 5 final_holdout_test  89      11
```

2.2.6 Further data exploration using visualisation

2.2.6.1 Disease proportion Proportion of those with Disease (ie =1) in the dataset is imbalanced - there are more cases with Disease than those without.

```
mean(train_set$Disease=="Yes")
```

```
## [1] 0.7159905
```

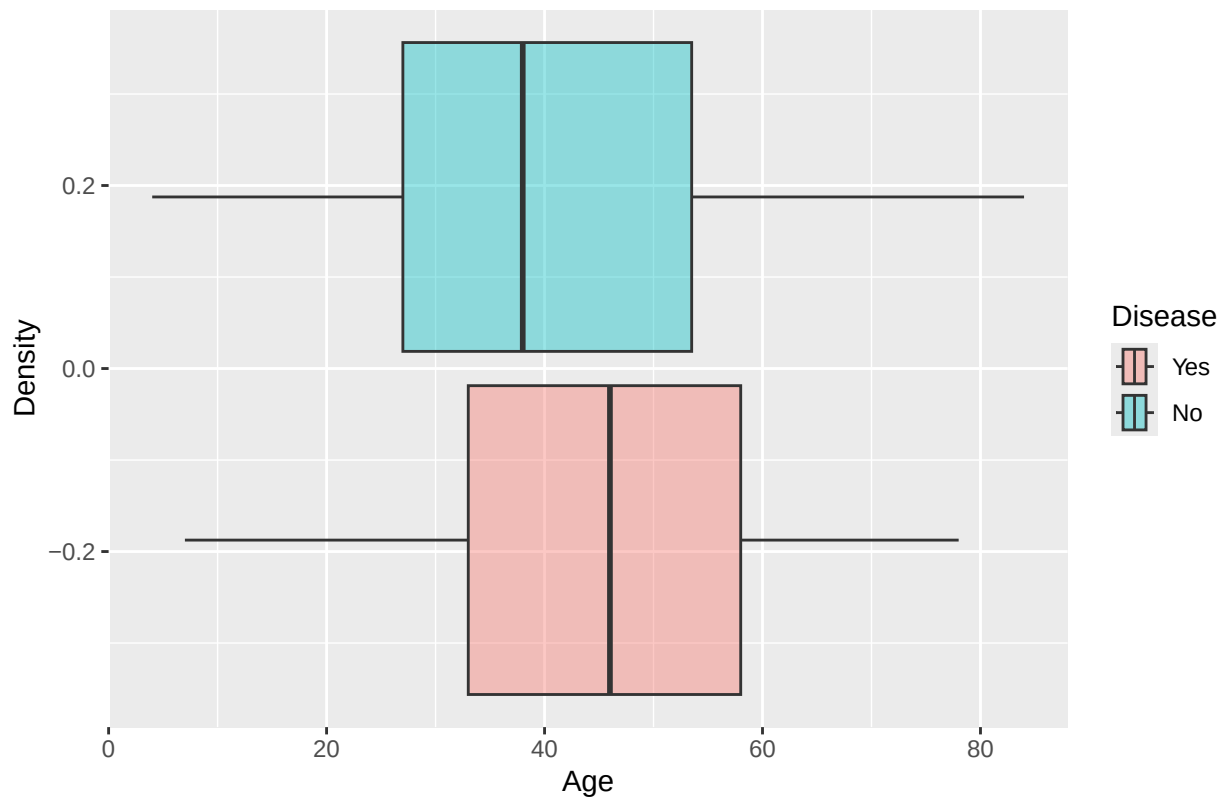
2.2.6.2 Age Older the higher the chance of having the disease but no disease group has wider range than

Age Distribution by Disease Status



the disease group.

Age Distribution by Disease Status (Boxplot)



Gender More males 74% over females 63% with disease. Similar to Age, it would be useful but there is overlap.

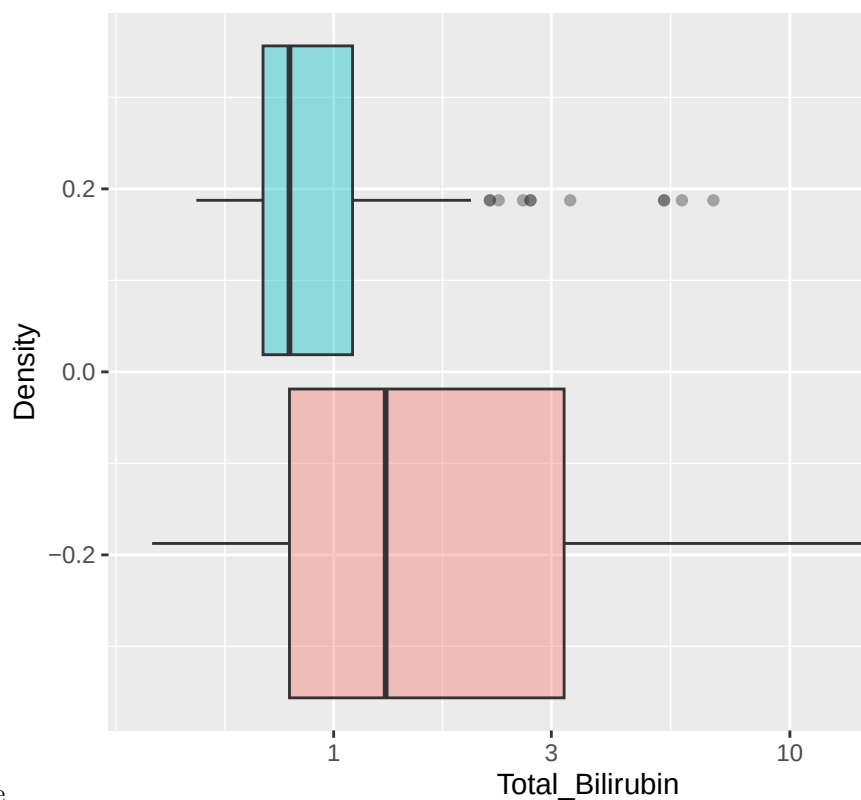
#Gender and its relationship to Disease

```
train_set %>%
  count(Gender, Disease) %>%
  group_by(Gender) %>%
  mutate(prop = n / sum(n))
```

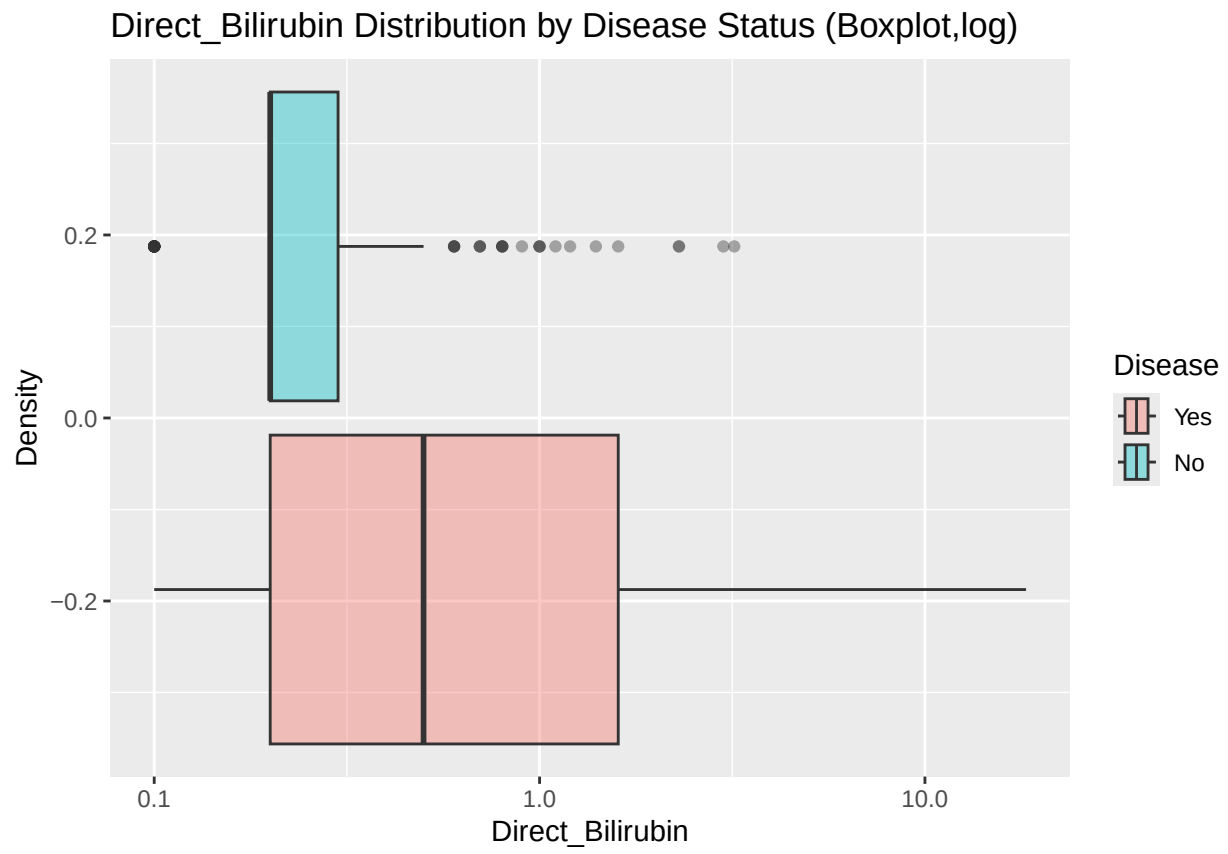
```
## # A tibble: 4 x 4
## # Groups:   Gender [2]
##   Gender Disease     n prop
##   <fct>   <fct> <int> <dbl>
## 1 Female  Yes     65 0.637
## 2 Female  No      37 0.363
## 3 Male    Yes    235 0.741
## 4 Male    No     82 0.259
```

2.2.6.3 Bilirubins Both total and direct (which is total-indirect bilirubin) appear to be the best discrimina-

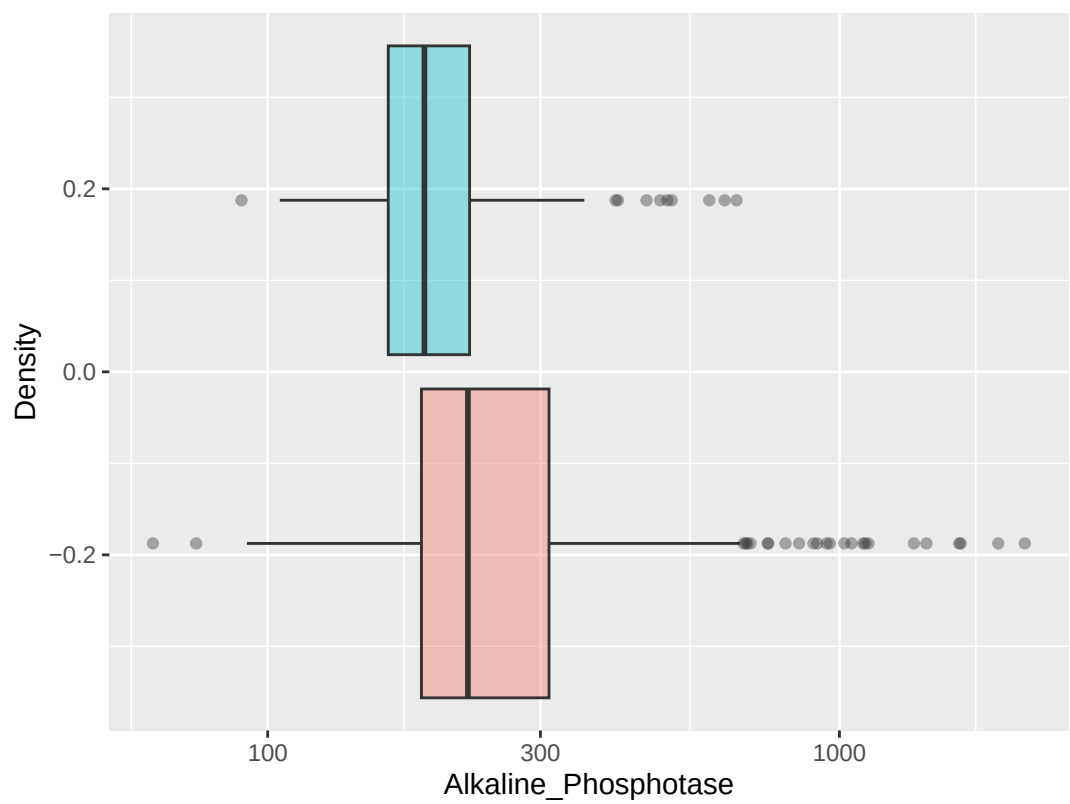
Total_Bilirubin Distribution by Disease Status (Boxplot)



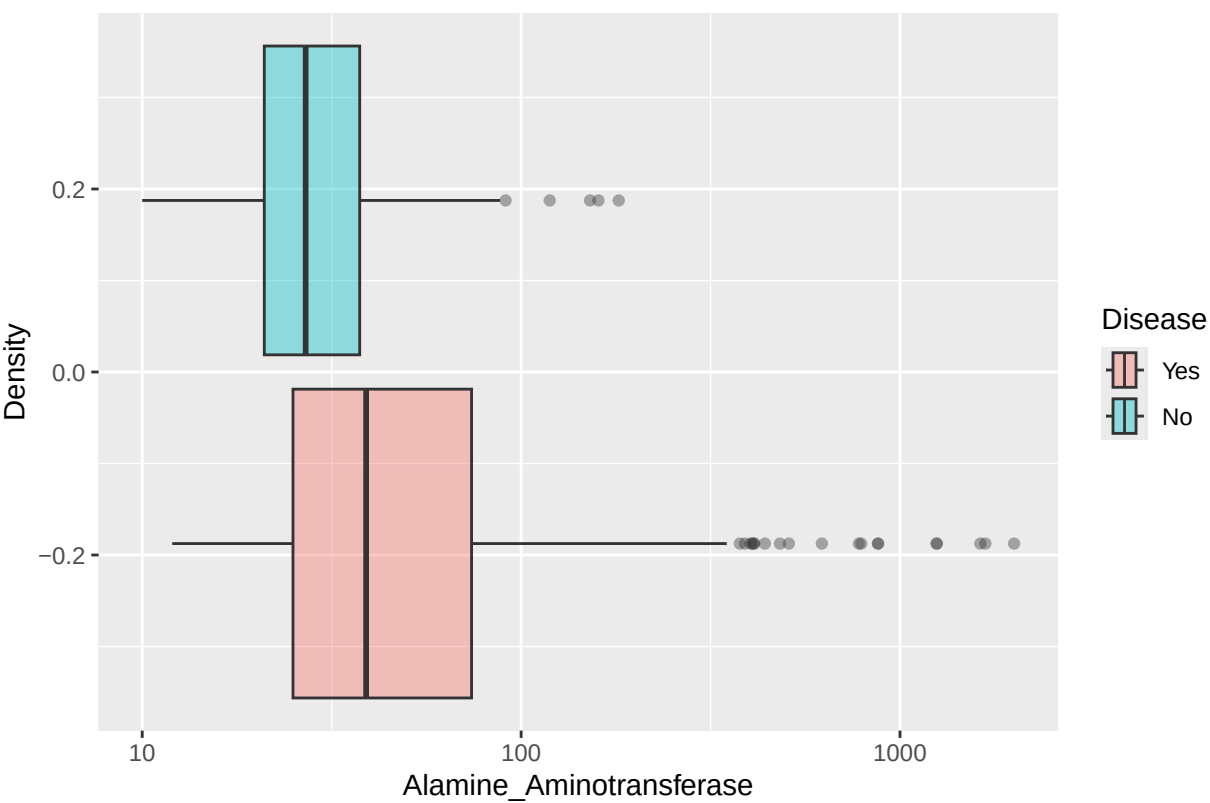
tor thus far. Lower the bili, less likely to have disease

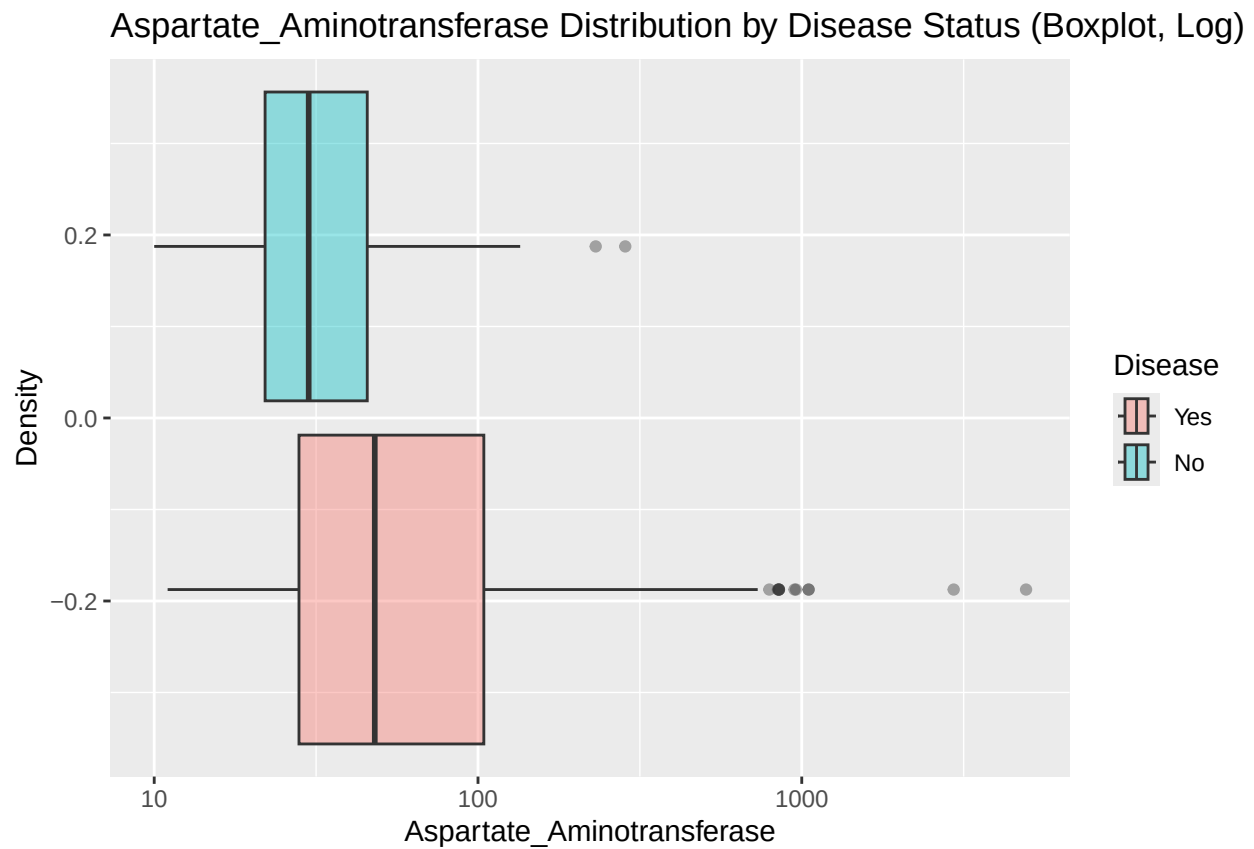


2.2.6.4 Liver enzymes (ALP, ALT, AST) All the enzymes show that higher the levels, higher the chance of having the Disease.
Alkaline_Phosphotase Distribution by Disease Status (Boxplot, Log)



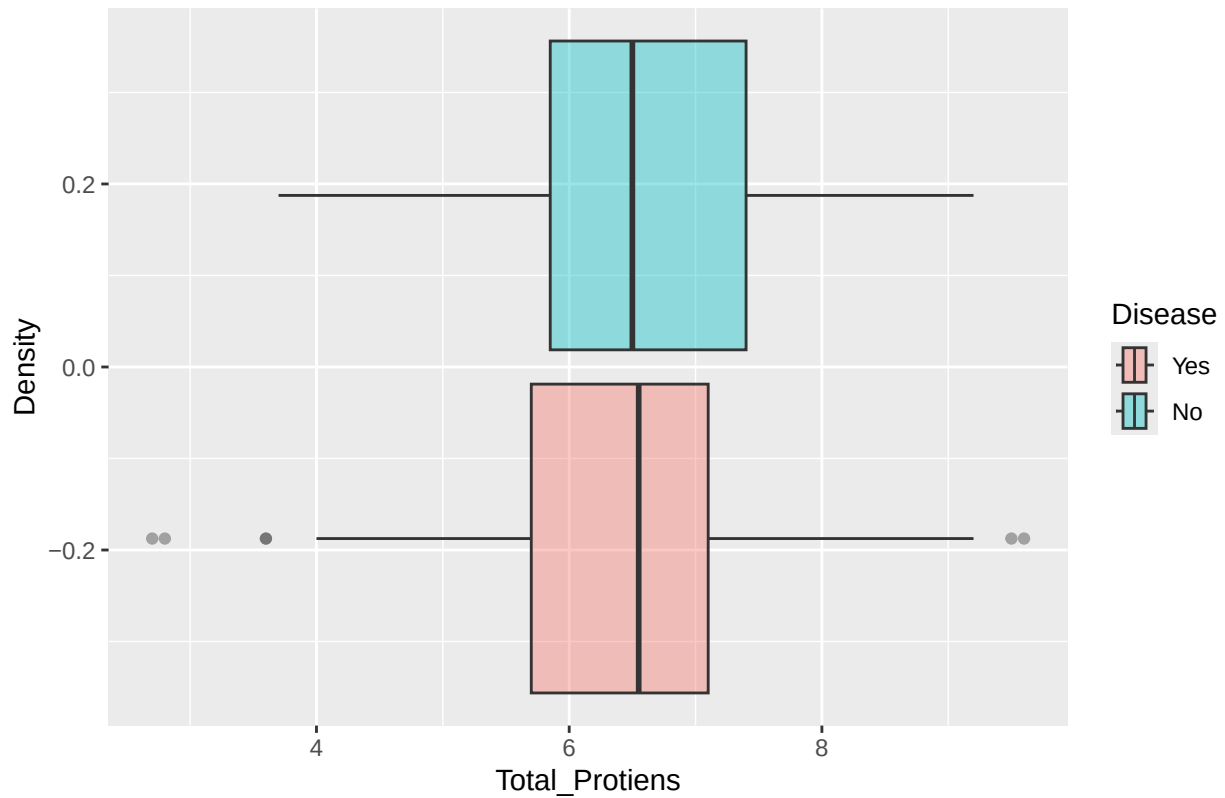
chance of having the Disease.
Alamine_Aminotransferase Distribution by Disease Status (Boxplot, Log)



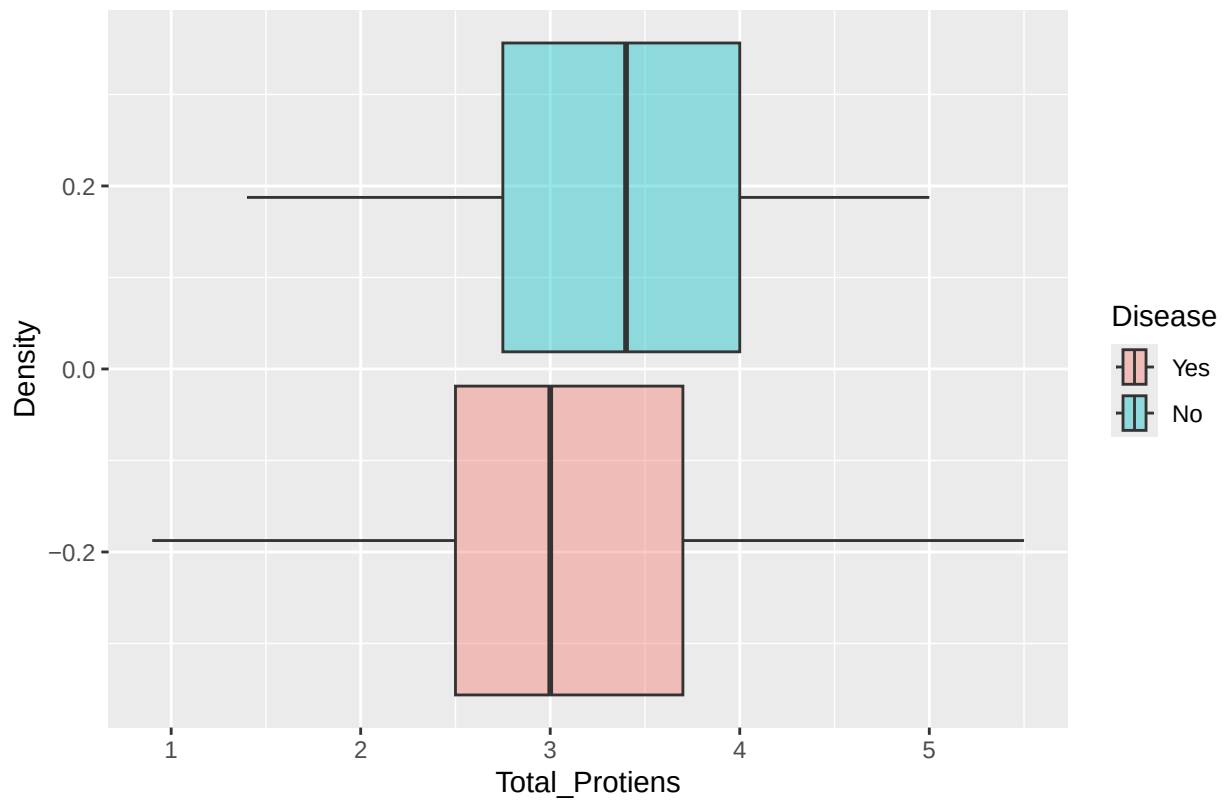


Protein related This reflects synthetic function of the liver. Total protein out of the other 2 protein markers (Albumin and Albumin Globulin ratio (AGR)) related features demonstrated best median difference.

Total_Protiens Distribution by Disease Status (Boxplot)

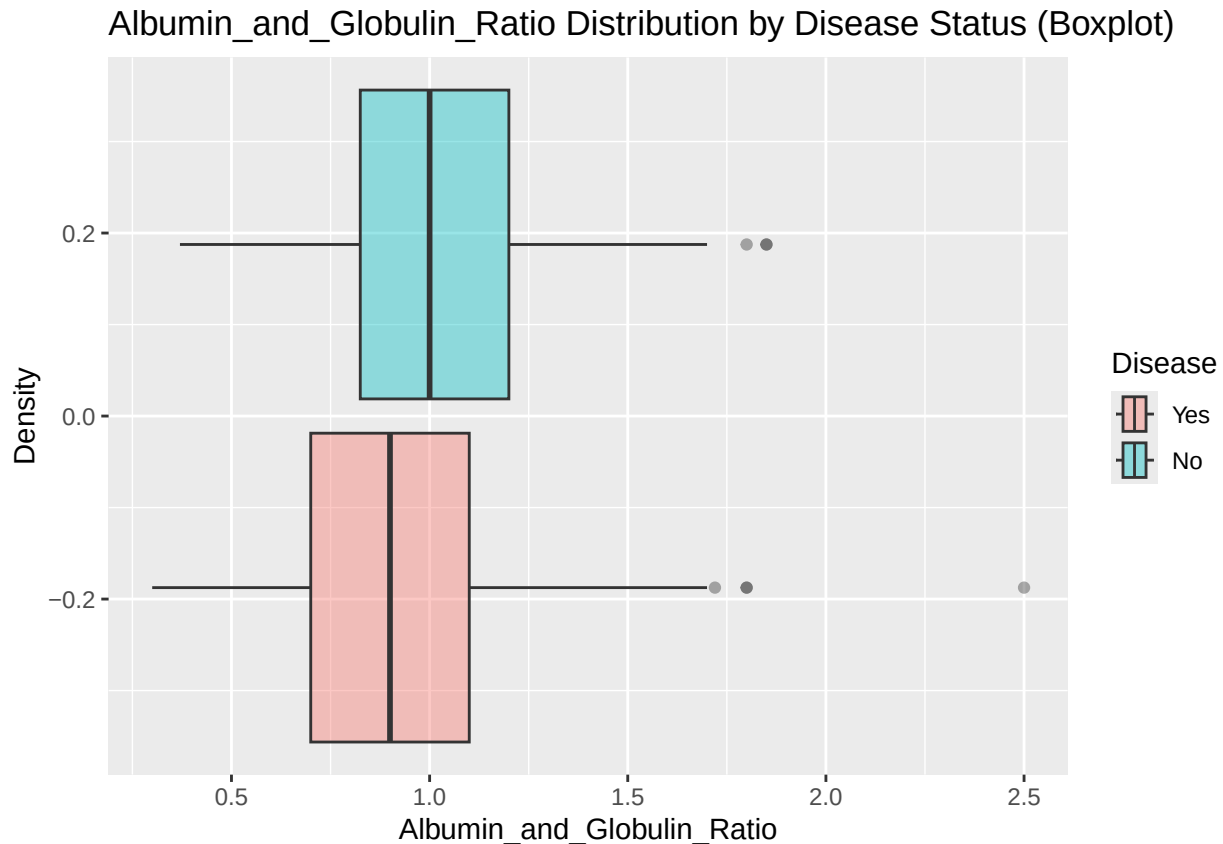


Total_Protiens Distribution by Disease Status (Boxplot)



Warning: Removed 2 rows containing non-finite outside the scale range

```
## (`stat_boxplot()`).
```



Summary of Exploration Above exploration shows that 1. Bilirubin and liver enzyme results likely carry the most weight in terms of prediction.

2. In the train data set, those with Disease was 'r $299/(299+118)*100\%$ (299 cases). This means there is imbalance of data used for binary categorisation. This would favour use of balanced accuracy over standard accuracy. Additionally, F1 score metric may be useful given that positive cases are important aspects of this machine learning tasks.
3. There are only 4 NA values in this dataset, in Albumin-Globulin ratio which overlaps with one another significantly.

3 Results

3.1 Handling of NA

Decision to remove 4 NA values within Albumin-Globulin Ratio instead of imputing the given that there are only 4 NA values.

```
train_set <- na.omit(train_set)
test_set <- na.omit(test_set)
```

3.2 Model 1: logistic regression (glm)

```
# train on train_set
train_glm <- train(Disease ~ ., method = "glm", data = train_set)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
#predict on test set based on train set
y_hat_glm <- predict(train_glm, test_set, type = "raw")

#metric calculation
cm_glm <- confusionMatrix(y_hat_glm, test_set$Disease)

cm_glm$byClass[["Balanced Accuracy"]]
cm_glm$byClass[["F1"]]
cm_glm$overall[["Accuracy"]]
```

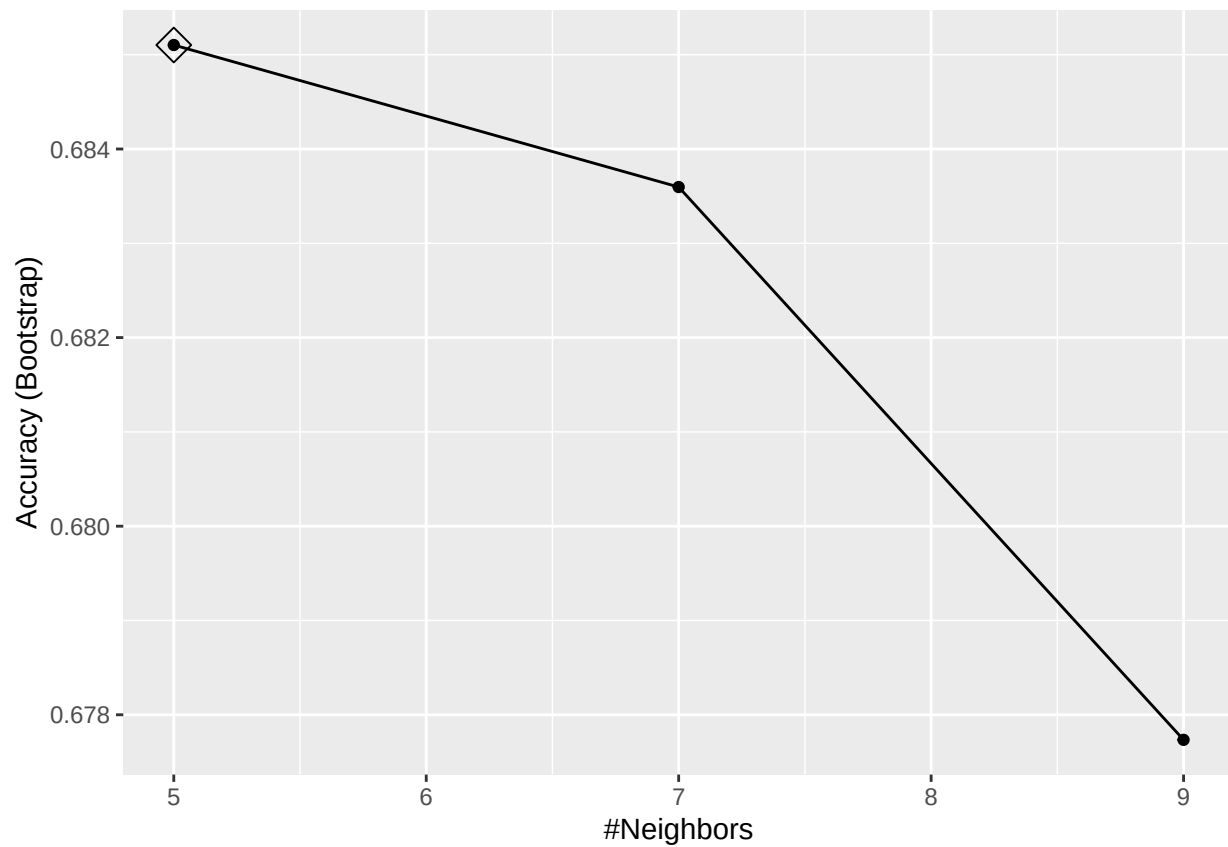
3.3 Model 2: KNN

```
#similar to previous - train on train set and then test on test set
train_knn <- train(Disease ~ ., method = "knn", data = train_set)
y_hat_knn <- predict(train_knn, test_set, type = "raw")
cm_knn <- confusionMatrix(y_hat_knn, test_set$Disease)

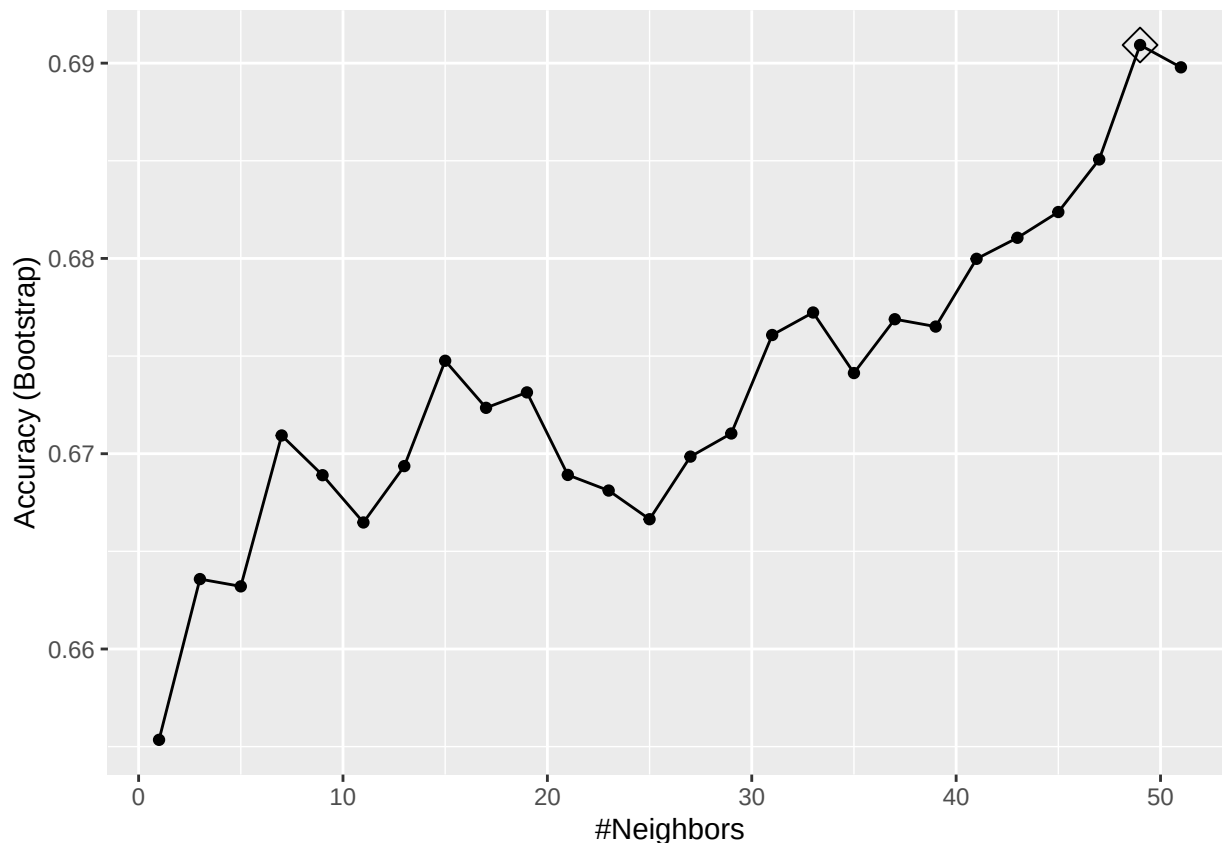
cm_knn$byClass[["Balanced Accuracy"]]
cm_knn$byClass[["F1"]]
cm_knn$overall[["Accuracy"]]

# knn tuning on trainset - there are <500 observations, and hence decided to test up to 51 k values
ggplot(train_knn, highlight = TRUE) #5-9 has been performed which achieves 0.6849315

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()``.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the caret package.
## Please report the issue at <https://github.com/topepo/caret/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



```
train_knn <- train(Disease ~ ., method = "knn",  
  data = train_set,  
  tuneGrid = data.frame(k = seq(1, 51, 2)))  
  
ggplot(train_knn, highlight = TRUE)
```



```
#best k=49
train_knn$bestTune
train_knn$finalModel

#test set
cm_knn <- confusionMatrix(data = predict(train_knn, test_set, type = "raw"),
                           reference = test_set$Disease)

cm_knn$byClass[["Balanced Accuracy"]]
cm_knn$byClass[["F1"]]
cm_knn$overall[["Accuracy"]]
```

3.4 Model 3: gamLoess

As some features may be non-linear, trial of gamloess

```
if(!require(gam)) install.packages("gam", repos = "http://cran.us.r-project.org")
modelLookup("gamLoess")
```

```
##      model parameter  label forReg forClass probModel
## 1 gamLoess      span   Span   TRUE    TRUE    TRUE
## 2 gamLoess    degree Degree   TRUE    TRUE    TRUE
```

```
#trying to find best span for the model with tuneGrid
```

```
#train control = 10 fold validations - same cross-validation folds
train_control <- trainControl(
  method = "cv",
```

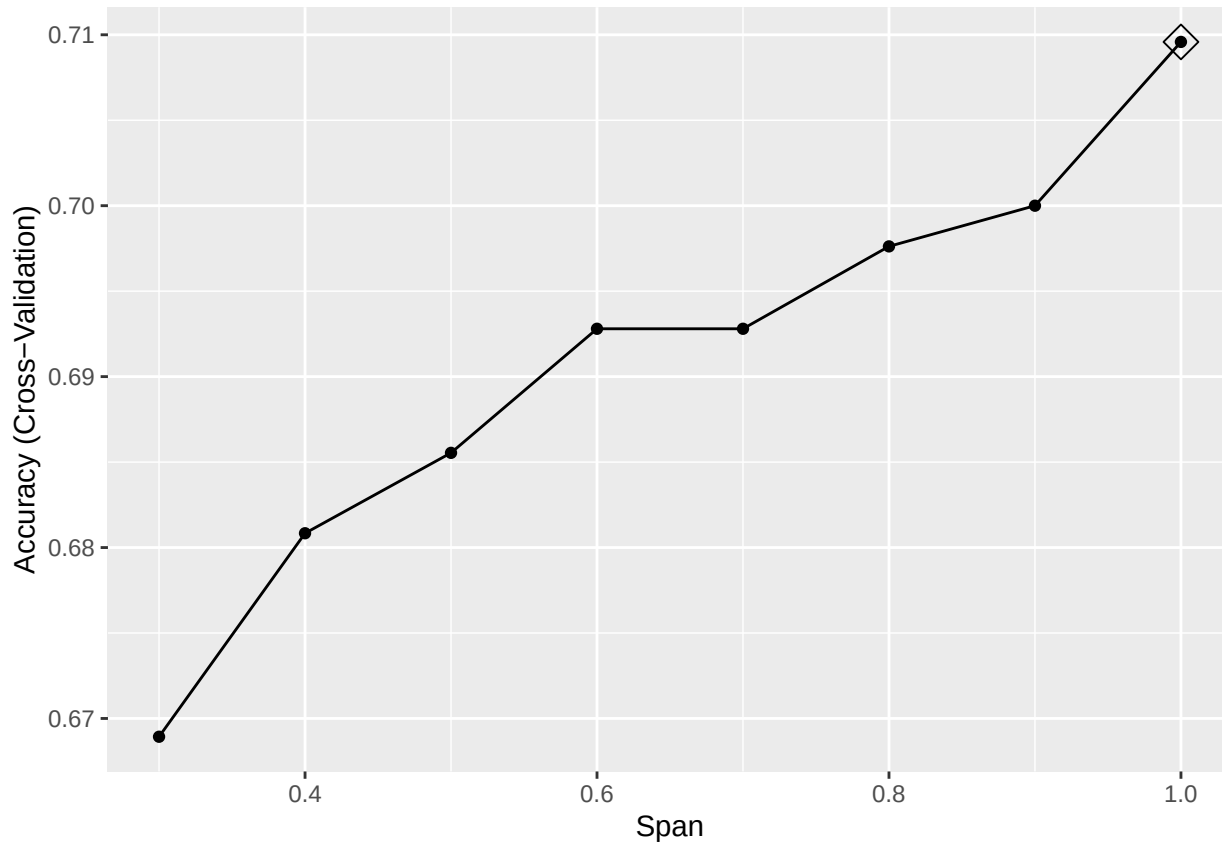
```

number = 10
)

#this will take about 30 seconds
train_gam <- train(Disease ~ ., data = train_set, method = "gamLoess",
  trControl = train_control,
  tuneGrid = data.frame(span = seq(0.3, 1.0, by = 0.1), degree = 1)
)

ggplot(train_gam, highlight = TRUE)

```



```

#predict on test set based on train set
y_hat_gam <- predict(train_gam, test_set)

#metric calculation
cm_gam <- confusionMatrix(y_hat_gam, test_set$Disease)

cm_gam$byClass[["Balanced Accuracy"]]

## [1] 0.6423993
cm_gam$byClass[["F1"]]

## [1] 0.8392857
cm_gam$overall[["Accuracy"]]

## [1] 0.7534247

```

3.5 Model 4: Random Forest

```
if(!require(randomForest)) install.packages("randomForest", repos = "http://cran.us.r-project.org")

## Loading required package: randomForest
## Warning: package 'randomForest' was built under R version 4.4.1
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
## The following object is masked from 'package:dplyr':
##
##      combine
## The following object is masked from 'package:ggplot2':
##
##      margin
#mtry values of c(2, 4, 6, 8) given about 10 features being assessed.

train_rf <- train(Disease ~ ., data = train_set, method = "rf",
  trControl = train_control, #train control = 10 fold validations previously defined
  tuneGrid = data.frame(
    mtry = c(2, 4, 6, 8)
  )
)

y_hat_rf <- predict(train_rf, test_set)

#metric calculation
cm_rf <- confusionMatrix(y_hat_rf, test_set$Disease)

cm_rf$byClass[["Balanced Accuracy"]]
cm_rf$byClass[["F1"]]
cm_rf$overall[["Accuracy"]]
```

3.6 Summary table of individual models

This shows that gamLoess achieves best balanced accuracy of 0.6423993 but random forest achieves best F1 value of 0.8521739.

```
cm_glm$byClass["Pos Pred Value"]

results <- data.frame(
  Model = c("Logistic Regression", "KNN", "gamLoess", "Random Forest"),
  Precision = c(
    cm_glm$byClass["Pos Pred Value"],
    cm_knn$byClass["Pos Pred Value"],
    cm_gam$byClass["Pos Pred Value"],
    cm_rf$byClass["Pos Pred Value"]
  ),
  Recall = c(
    cm_glm$byClass["Sensitivity"],
```



```

    cm_knn$byClass["Sensitivity"],
    cm_gam$byClass["Sensitivity"],
    cm_rf$byClass["Sensitivity"]
  ),
  Specificity = c(
    cm_glm$byClass["Specificity"],
    cm_knn$byClass["Specificity"],
    cm_gam$byClass["Specificity"],
    cm_rf$byClass["Specificity"]
  ),
  F1 = c(
    cm_glm$byClass["F1"],
    cm_knn$byClass["F1"],
    cm_gam$byClass["F1"],
    cm_rf$byClass["F1"]
  ),
  Balanced_Accuracy = c(
    cm_glm$byClass["Balanced Accuracy"],
    cm_knn$byClass["Balanced Accuracy"],
    cm_gam$byClass["Balanced Accuracy"],
    cm_rf$byClass["Balanced Accuracy"]
  ),
  Accuracy = c(
    cm_glm$overall["Accuracy"],
    cm_knn$overall["Accuracy"],
    cm_gam$overall["Accuracy"],
    cm_rf$overall["Accuracy"]
  )
)
results

```

3.7 Ensemble with stacked models

```

# Use caretEnsemble package
if(!require(caretEnsemble)) install.packages("caretEnsemble", repos = "http://cran.us.r-project.org")

# New model list creation combining all models with specific commands:

train_control <- trainControl(
  method = "cv",
  number = 10,
  savePredictions = "final"
)

model_list <- caretList(
  Disease ~ .,
  data = train_set,
  trControl = train_control,
  tuneList = list(
    glm = caretModelSpec(method = "glm"),
    knn = caretModelSpec(
      method = "knn",

```

```

    tuneGrid = data.frame(k = seq(1, 51, 2)),
    preProcess = c("center", "scale")
  ),
  gam = caretModelSpec(
    method = "gamLoess",
    tuneGrid = expand.grid(
      span = seq(0.3, 1, 0.1),
      degree = 1
    )
  ),
  rf = caretModelSpec(
    method = "rf",
    tuneGrid = data.frame(mtry = c(2, 4, 6, 8))
  )
)

# stacking using glm
stack_model <- caretStack(
  model_list,
  method = "glm"
)

#y_hat prediction
y_hat_ensemble <- predict(stack_model, newdata = test_set)

#Transform this to a factored outcome
y_hat_class <- ifelse(
  y_hat_ensemble$Yes >= 0.5,
  "Yes",
  "No"
)

y_hat_class <- factor(
  y_hat_class,
  levels = levels(test_set$Disease)
)

cm_ensemble <- confusionMatrix(
  y_hat_class,
  test_set$Disease,
  positive = "Yes"
)

cm_ensemble

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Yes No
##           Yes  47 12
##           No   5  9
##
##           Accuracy : 0.7671

```

```
##          95% CI : (0.6535, 0.8581)
##    No Information Rate : 0.7123
##    P-Value [Acc > NIR] : 0.1836
##
##          Kappa : 0.3691
##
##    McNemar's Test P-Value : 0.1456
##
##          Sensitivity : 0.9038
##          Specificity : 0.4286
##    Pos Pred Value : 0.7966
##    Neg Pred Value : 0.6429
##          Prevalence : 0.7123
##    Detection Rate : 0.6438
##    Detection Prevalence : 0.8082
##    Balanced Accuracy : 0.6662
##
##    'Positive' Class : Yes
##
```

```
results <- bind_rows(results,
  tibble(Model = "Ensemble of 4 Models",
    Precision = cm_ensemble$byClass["Pos Pred Value"],
    Recall = cm_ensemble$byClass["Sensitivity"],
    Specificity = cm_ensemble$byClass["Specificity"],
    F1 = cm_ensemble$byClass["F1"],
    Balanced_Accuracy = cm_ensemble$byClass["Balanced Accuracy"],
    Accuracy = cm_ensemble$overall["Accuracy"]))

results
```

	Model	Precision	Recall	Specificity	F1
## 1	Logistic Regression	0.7666667	0.8846154	0.3333333	0.8214286
## 2	KNN	0.7323944	1.0000000	0.0952381	0.8455285
## 3	gamLoess	0.7833333	0.9038462	0.3809524	0.8392857
## 4	Random Forest	0.7833333	0.9038462	0.3809524	0.8392857
## 5	Ensemble of 4 Models	0.7966102	0.9038462	0.4285714	0.8468468
##	Balanced_Accuracy	Accuracy			
## 1		0.6089744	0.7260274		
## 2		0.5476190	0.7397260		
## 3		0.6423993	0.7534247		
## 4		0.6423993	0.7534247		
## 5		0.6662088	0.7671233		

Across the models, the precision and recall rates were high (0.75 to 0.95) with a low specificity rates, mostly in 0.30-0.38 range. Ensemble's specificity rate was highest at 0.42, which translated to the best balanced accuracy.

F1 score was highest for Random forest, thanks to its highest recall and third highest precision rate.

It appears that ensemble achieves the best metric outcome for precision, specificity and all balanced accuracy. Random forest followed closely and achieves best recall, but lower specificity with lower balanced accuracy. gamLoess had the second highest balanced accuracy with comparable recall to ensemble but lower specificity.

3.8 Test on final hold out set

These models were then tested on the final hold out set as below.

```
# glm
y_hat_glm_2 <- predict(train_glm, final_holdout_test, type = "raw")
cm_glm_2 <- confusionMatrix(y_hat_glm_2, final_holdout_test$Disease)

cm_glm_2

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Yes No
##          Yes  57 21
##          No   6  5
##
##              Accuracy : 0.6966
##              95% CI : (0.5901, 0.7897)
##          No Information Rate : 0.7079
##          P-Value [Acc > NIR] : 0.641909
##
##              Kappa : 0.1169
##
##  Mcnemar's Test P-Value : 0.007054
##
##              Sensitivity : 0.9048
##              Specificity : 0.1923
##              Pos Pred Value : 0.7308
##              Neg Pred Value : 0.4545
##              Prevalence : 0.7079
##              Detection Rate : 0.6404
##          Detection Prevalence : 0.8764
##              Balanced Accuracy : 0.5485
##
##              'Positive' Class : Yes
##
# knn
cm_knn_2 <- confusionMatrix(data = predict(train_knn, final_holdout_test, type = "raw"),
                           reference = final_holdout_test$Disease)

# gamLoess
y_hat_gam_2 <- predict(train_gam, final_holdout_test)
cm_gam_2 <- confusionMatrix(y_hat_gam_2, final_holdout_test$Disease)

# rf
y_hat_rf_2 <- predict(train_rf, final_holdout_test)
cm_rf_2 <- confusionMatrix(y_hat_rf_2, final_holdout_test$Disease)

# ensemble
y_hat_ensemble_2 <- predict(stack_model, newdata = final_holdout_test)

#Transform this to a factored outcome
y_hat_class_2 <- ifelse(
  y_hat_ensemble_2$Yes >= 0.5,
  "Yes",
```

```

    "No"
  )

y_hat_class_2 <- factor(
  y_hat_class_2,
  levels = levels(final_holdout_test$Disease)
)

cm_ensemble_2 <- confusionMatrix(
  y_hat_class_2,
  final_holdout_test$Disease,
  positive = "Yes"
)

cm_ensemble_2

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Yes No
##           Yes  56 22
##           No   7  4
##
##           Accuracy : 0.6742
##           95% CI : (0.5666, 0.7698)
##           No Information Rate : 0.7079
##           P-Value [Acc > NIR] : 0.79423
##
##           Kappa : 0.0515
##
## Mcnemar's Test P-Value : 0.00933
##
##           Sensitivity : 0.8889
##           Specificity : 0.1538
##           Pos Pred Value : 0.7179
##           Neg Pred Value : 0.3636
##           Prevalence : 0.7079
##           Detection Rate : 0.6292
##           Detection Prevalence : 0.8764
##           Balanced Accuracy : 0.5214
##
##           'Positive' Class : Yes
##

results_2 <- data.frame(
  Model = c("Logistic Regression", "KNN", "gamLoess", "Random Forest", "Ensemble of 4 Models"),
  Precision = c(
    cm_glm_2$byClass["Pos Pred Value"],
    cm_knn_2$byClass["Pos Pred Value"],
    cm_gam_2$byClass["Pos Pred Value"],
    cm_rf_2$byClass["Pos Pred Value"],
    cm_ensemble_2$byClass["Pos Pred Value"]
  ),
  Recall = c(

```

```

cm_glm_2$byClass["Sensitivity"],
cm_knn_2$byClass["Sensitivity"],
cm_gam_2$byClass["Sensitivity"],
cm_rf_2$byClass["Sensitivity"],
cm_ensemble_2$byClass["Sensitivity"]
),
Specificity = c(
cm_glm_2$byClass["Specificity"],
cm_knn_2$byClass["Specificity"],
cm_gam_2$byClass["Specificity"],
cm_rf_2$byClass["Specificity"],
cm_ensemble_2$byClass["Specificity"]
),
F1 = c(
cm_glm_2$byClass["F1"],
cm_knn_2$byClass["F1"],
cm_gam_2$byClass["F1"],
cm_rf_2$byClass["F1"],
cm_ensemble_2$byClass["F1"]
),
Balanced_Accuracy = c(
cm_glm_2$byClass["Balanced Accuracy"],
cm_knn_2$byClass["Balanced Accuracy"],
cm_gam_2$byClass["Balanced Accuracy"],
cm_rf_2$byClass["Balanced Accuracy"],
cm_ensemble_2$byClass["Balanced Accuracy"]
),
Accuracy = c(
cm_glm_2$overall["Accuracy"],
cm_knn_2$overall["Accuracy"],
cm_gam_2$overall["Accuracy"],
cm_rf_2$overall["Accuracy"],
cm_ensemble_2$overall["Accuracy"]
)
)

results_2

```

##		Model	Precision	Recall	Specificity	F1
## 1	Logistic Regression	0.7307692	0.9047619	0.19230769	0.8085106	
## 2	KNN	0.7142857	0.9523810	0.07692308	0.8163265	
## 3	gamLoess	0.7307692	0.9047619	0.19230769	0.8085106	
## 4	Random Forest	0.7142857	0.8730159	0.15384615	0.7857143	
## 5	Ensemble of 4 Models	0.7179487	0.8888889	0.15384615	0.7943262	
##	Balanced_Accuracy	Accuracy				
## 1		0.5485348	0.6966292			
## 2		0.5146520	0.6966292			
## 3		0.5485348	0.6966292			
## 4		0.5134310	0.6629213			
## 5		0.5213675	0.6741573			

The result shows that the models overall have high precision and recall, resulting in high F1 scores. However due to low specificity between 10-20%, this results in low balanced accuracy rating. Random forest had highest precision, specificity and second highest recall, placing it at the highest balanced accuracy of 0.560.

4 Conclusion

This project tested machine learning algorithms on Indian Liver Disease database to come up with categorisation algorithm. Using machine learning techniques, we were able to identify that random forest model achieved overall best metric. This was closely followed by logistic regression, gamLoess, and the Ensemble of the 4 models. Future work could look into trial of other machine learning techniques including receiver operating curve.

In summary, use of machine learning methods achieved categorisation algorithms with high recall (sensitivity) but with low specificity. This means that the test rules out when the test is negative (high sensitivity) but the converse is not true - that is when the test is positive, it is not able to rule in the disease. This would mean that the test is potentially suitable for screening for a large population, and further work is needed to develop better test which can confirm the disease in those who test positive on this test (given its low specificity).

5 References

1. Friedman LS. Approach to the patient with abnormal liver tests. UpToDate. 2026.
2. Devarbhavi H, Asrani SK, Arab JP, Nartey YA, Pose E, Kamath PS. Global burden of liver disease: 2023 update. J Hepatol. 2023 Aug;79(2):516-537. doi: 10.1016/j.jhep.2023.03.017. Epub 2023 Mar 27.
3. Large language model, ChatGPT, was used for checking for coding errors.